

PYTHON AUTOMATION – LAB ASSIGNMENTS

MODULE 1: AUTOMATION WITH SELENIUM

Assignments 1–6 (Tier 1 & Tier 2)

Name: Swarnendu Boral

Roll No : 10

Section: CSE(AI)

Enrollment Number: 12023052020011

Overview

This document solves the six official Module 1 lab-assignment options listed in the syllabus (Tier 1: Core Fundamentals & Locators — Assignments 1 to 3; Tier 2: Advanced User Interactions — Assignments 4 to 6). Each assignment follows the same record format used in the Module 1 lab workbook.

Practice sites used: SauceDemo (saucedemo.com), the-internet.herokuapp.com, and SeleniumEasy.com (seleniumeasy.com) — all standard, publicly available Selenium practice applications.

Assignment 1: The Multi-Locator Challenge

Module	Module 1 – Automation with Selenium (Lab Assignments)
Assignment Title	The Multi-Locator Challenge
Assignment Tier	Tier 1 – Core Fundamentals & Locators

1. Problem Statement

Navigate to a login page (SauceDemo) and interact with its fields using three different locator strategies in a single flow, then confirm a successful login by checking the resulting page URL.

2. Objective

- Locate the username field using By.ID.
- Locate the password field using By.NAME.
- Locate the login button using By.XPATH.
- Assert that the resulting URL contains '/inventory.html' after a successful login.

3. Tools, Software & Concepts Used

- Selenium 4, selenium.webdriver.common.by.By
- Practice site: <https://www.saucedemo.com/>
- Concepts: mixed locator strategies within one flow, post-login URL assertion

4. Implementation Steps

1. Open <https://www.saucedemo.com/>.
2. Locate the username field with By.ID ('user-name') and enter a valid username.
3. Locate the password field with By.NAME ('password') and enter the matching password.
4. Locate the login button with By.XPATH and click it.
5. Assert that '/inventory.html' is present in driver.current_url.

5. Source Code

```
# assignment1_multi_locator_challenge.py

from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get('https://www.saucedemo.com/')

# 1. Username field -> By.ID
username_field = driver.find_element(By.ID, 'user-name')
username_field.send_keys('standard_user')
```

```
# 2. Password field -> By.NAME
password_field = driver.find_element(By.NAME, 'password')
password_field.send_keys('secret_sauce')

# 3. Login button -> By.XPATH
login_button = driver.find_element(By.XPATH, "//input[@id='login-button']")
login_button.click()

# 4. Validation
assert '/inventory.html' in driver.current_url, (
    f'Login failed or redirected unexpectedly. Current URL: {driver.current_url}'
)
print('Login successful. Current URL:', driver.current_url)

driver.quit()
```

6. Output

The script logs into SauceDemo using three distinct locator strategies for the three fields/controls involved, then prints the post-login URL once the assertion passes.

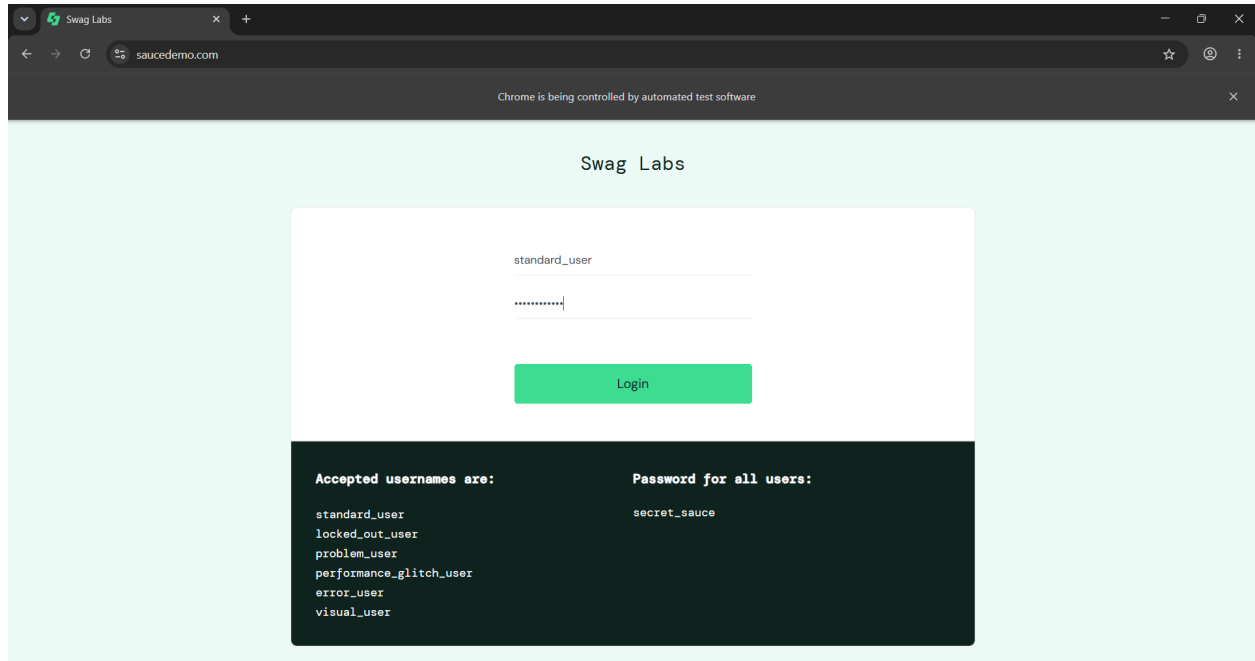


Fig A1.1 — SauceDemo login page before submission, with username/password filled in.

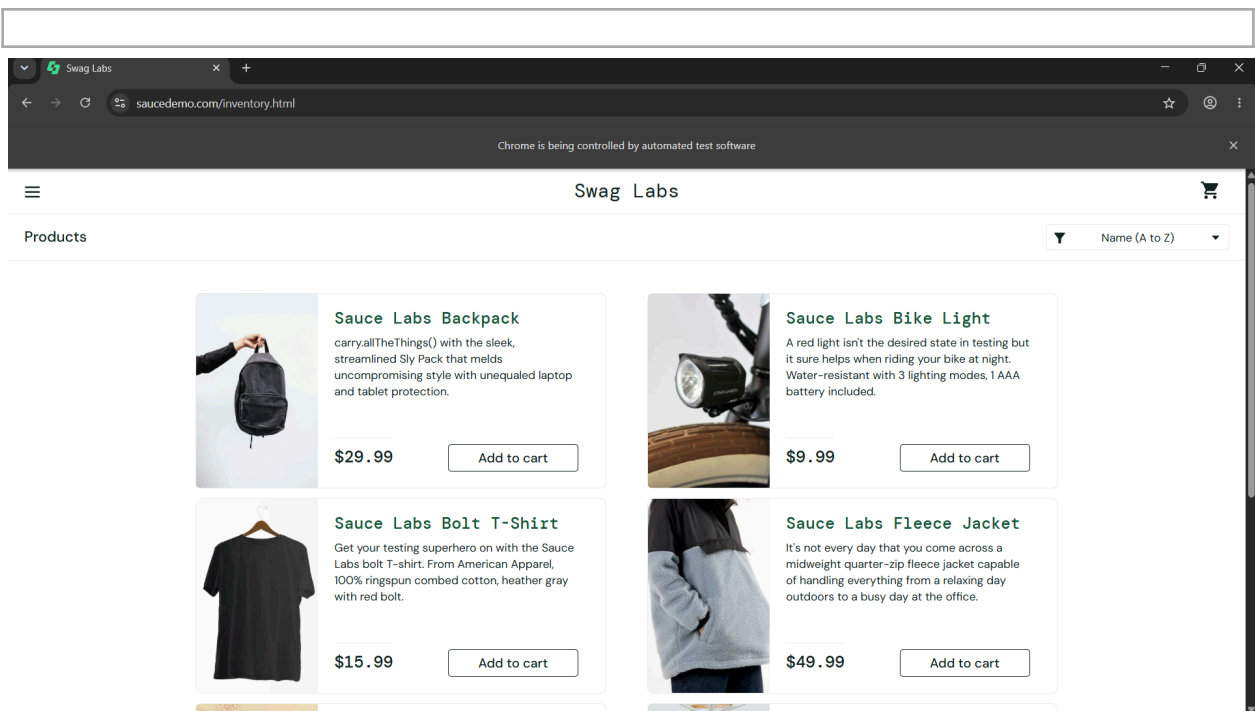


Fig A1.2 — Inventory page reached after login, confirming '/inventory.html' in the URL bar.

7. Result

Login was completed successfully using By.ID, By.NAME and By.XPATH together, and the URL assertion passed.

8. Observation

- SauceDemo's username field happens to share the same value for both its id and name attributes, so By.ID and By.NAME would both have worked here — the assignment deliberately assigns one strategy per field to force practice with each locator type rather than defaulting to the easiest one.
- Asserting on a URL fragment (rather than a visible heading or welcome text) is a more robust way to confirm navigation, since visible text can change across app versions while route paths are more stable.

9. Conclusion

Three different locator strategies were combined in a single realistic login flow, reinforcing that locator choice is a per-element decision rather than a single fixed strategy for an entire script.

Assignment 2: Synchronization & Explicit Waits

Module	Module 1 – Automation with Selenium (Lab Assignments)
Assignment Title	Synchronization & Explicit Waits
Assignment Tier	Tier 1 – Core Fundamentals & Locators

1. Problem Statement

A page reveals text only after a multi-second loading animation completes. The script must wait for that exact moment without using a fixed delay, since the load time is not perfectly predictable.

2. Objective

- Click a 'Start' control that triggers a delayed reveal of hidden text.
- Wait for the revealed element using WebDriverWait + expected_conditions only — no time.sleep() anywhere in the script.
- Extract and print the revealed text once it becomes visible.

3. Tools, Software & Concepts Used

- selenium.webdriver.support.ui.WebDriverWait
- selenium.webdriver.support.expected_conditions (EC)
- Practice site: https://the-internet.herokuapp.com/dynamic_loading/1
- Concepts: explicit wait, expected_conditions.visibility_of_element_located

4. Implementation Steps

1. Open the dynamic-loading example page.
2. Click the 'Start' button, which triggers a ~5 second loading bar.
3. Create a WebDriverWait(driver, 10) instance.
4. Call .until(EC.visibility_of_element_located((By.ID, 'finish')) to block only until the element is visible.
5. Read and print the revealed element's text.

5. Source Code

```
# assignment2_explicit_wait.py
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.maximize_window()
driver.get('https://the-internet.herokuapp.com/dynamic_loading/1')
```

```

driver.find_element(By.CSS_SELECTOR, '#start button').click()

# No time.sleep() anywhere - purely condition-based waiting.
wait = WebDriverWait(driver, 10)
finish_element = wait.until(
    EC.visibility_of_element_located((By.ID, 'finish'))
)

print('Revealed text:', finish_element.text)
assert finish_element.text == 'Hello World!'

driver.quit()

```

6. Output

The script clicks Start, blocks only for as long as the loading bar actually takes, then prints the revealed text as soon as it becomes visible.

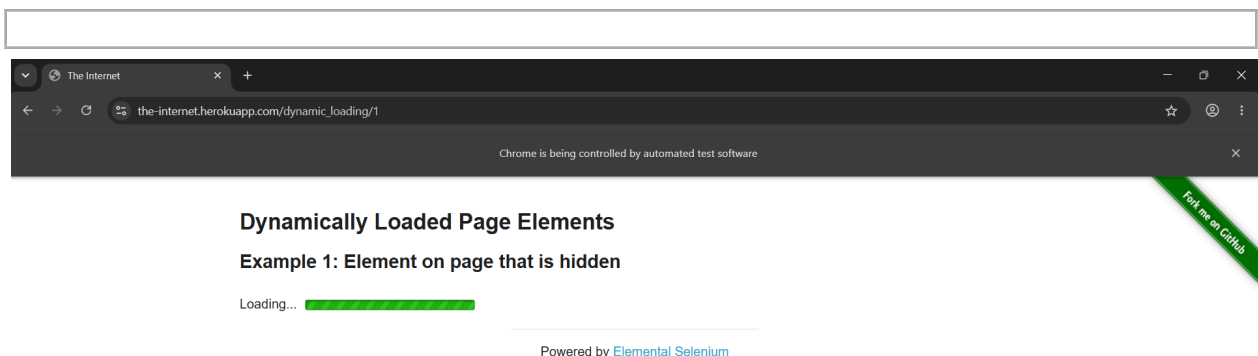


Fig A2.1 — Loading bar mid-animation immediately after clicking Start.

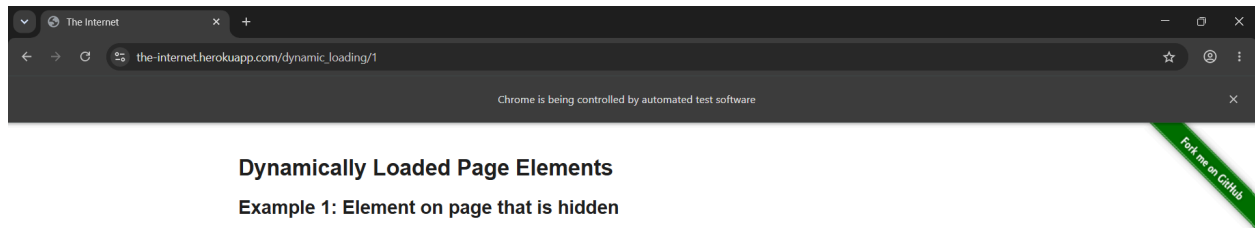


Fig A2.2 — 'Hello World!' text visible once the explicit wait condition resolves.

7. Result

The script synchronized correctly with the dynamic content, printing the revealed text with no hard-coded delay used.

8. Observation

- `visibility_of_element_located` checks both presence in the DOM and that the element is actually rendered (non-zero size, not hidden) — `presence_of_element_located` alone would have returned an element handle before the text was actually visible, since the `#finish` element exists in the DOM but is hidden pre-load.
- Because the wait polls repeatedly (default ~0.5s) rather than blocking for a fixed duration, the script returns as soon as the condition is true, making it both faster and more resilient to timing variance than `time.sleep(5)` would be.

9. Conclusion

Explicit, condition-based waiting proved to be the correct mechanism for handling delayed content, avoiding both premature failures and unnecessary fixed delays.

Assignment 3: Dynamic Dropdowns & Checkboxes

Module	Module 1 – Automation with Selenium (Lab Assignments)
Assignment Title	Dynamic Dropdowns & Checkboxes
Assignment Tier	Tier 1 – Core Fundamentals & Locators

1. Problem Statement

A page contains multiple checkboxes that must be selected with state verification, and a separate search-as-you-type autocomplete field whose suggestion list is generated dynamically. The script must handle both: deterministic static controls, and an asynchronous, dynamically rendered widget.

2. Objective

- Select multiple checkboxes on a form and verify each one's state with `.is_selected()`.
- Type a partial value into an autocomplete field and wait for the suggestion list to populate.
- Loop through the suggested results and click the one that matches the intended value.

3. Tools, Software & Concepts Used

- Practice sites: <https://the-internet.herokuapp.com/checkboxes> (checkboxes), <https://jqueryui.com/resources/demos/autocomplete/default.html> (official jQuery UI autocomplete demo)
- `selenium.webdriver.support.ui.WebDriverWait`
- Concepts: checkbox state verification, dynamic suggestion-list traversal

4. Implementation Steps

1. Open the checkboxes demo page and locate both `<input type='checkbox'>` elements.
2. Click each checkbox that is not already selected, then verify `.is_selected()` reports True for both.
3. Open the jQuery UI autocomplete demo page.
4. Click the `#tags` input and type a partial value (e.g., `'ja'`) to trigger the suggestion list.
5. Wait for the suggestion `<li>` items inside `'ul.ui-autocomplete'` to appear, loop through them, and click the one whose text exactly matches the target value (e.g., `'JavaScript'`).
6. Confirm the input field's final value equals the selected suggestion.

5. Source Code

```
# assignment3_dropdowns_checkboxes.py
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```

driver = webdriver.Chrome()
driver.maximize_window()
wait = WebDriverWait(driver, 10)

# --- 1. Multiple checkboxes with state verification ---
driver.get('https://the-internet.herokuapp.com/checkboxes')

checkboxes = driver.find_elements(By.CSS_SELECTOR, "input[type='checkbox']")
print(f'Total checkboxes found: {len(checkboxes)}')

for idx, cb in enumerate(checkboxes):
    if not cb.is_selected():
        cb.click()

    print(f'Checkbox {idx + 1} selected:', cb.is_selected())
    assert cb.is_selected() is True

# --- 2. Autocomplete: type + select matching suggestion from a dynamic list ---
driver.get('https://jqueryui.com/resources/demos/autocomplete/default.html')

target_tag = 'JavaScript'
search_box = driver.find_element(By.ID, 'tags')
search_box.click()
search_box.send_keys('ja')

suggestions = wait.until(
    EC.presence_of_all_elements_located((By.CSS_SELECTOR, 'ul.ui-autocomplete
li'))
)

matched = False
for suggestion in suggestions:
    if suggestion.text.strip() == target_tag:
        suggestion.click()

```

```

    matched = True
    break

assert matched, f"No suggestion matched '{target_tag}'"
print('Autocomplete field value after selection:',
      search_box.get_attribute('value'))

driver.quit()

```

6. Output

The script selects and verifies both checkboxes, then types a partial tag name, waits for the live suggestion list, loops through it, and clicks the exact match — printing the final field value.

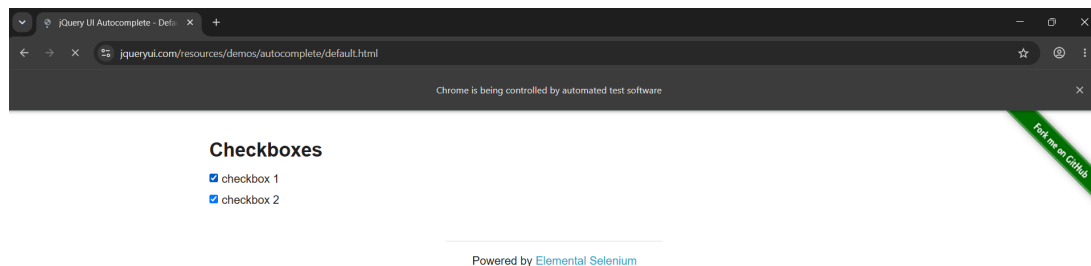


Fig A3.1 — Checkboxes demo page with both checkboxes checked.

```

C:\Users\swarn\AppData\Local\Microsoft\WindowsApps\python3.exe C:\Users\swarn\Downloads\my-course-repo\PythonProject\practice.py
Total checkboxes found: 2
Checkbox 1 selected: True
Checkbox 2 selected: True
Autocomplete field value after selection: JavaScript
Process finished with exit code 0

```

Fig A3.2 — Console output confirming True for each checkbox's `is_selected()` call.

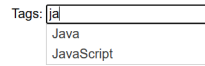


Fig A3.3 — Autocomplete suggestion list open after typing 'ja' (showing Java / JavaScript).

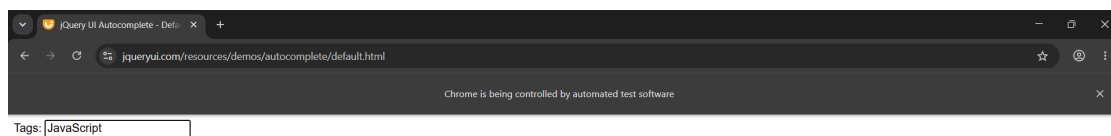


Fig A3.4 — Field showing 'JavaScript' after the matching suggestion is clicked.

7. Result

Checkbox selection was verified programmatically, and the autocomplete flow correctly matched and selected the intended suggestion out of a dynamically generated list.

8. Observation

- `is_selected()` reflects the live DOM state of the checkbox, so verifying it immediately after `.click()` is a reliable way to catch a click that silently failed to register.
- The autocomplete suggestion list does not exist in the DOM until the input triggers it, so `presence_of_all_elements_located` must run after typing, and an explicit wait is required since the list is populated by the widget's own JavaScript filtering logic asynchronously.

- Matching on `suggestion.text.strip() == target_tag` (exact match) avoids accidentally clicking a partial match like 'Java' when searching for 'JavaScript' — both appear in the same suggestion list for 'ja'.

9. Conclusion

Both static, page-load-time controls (checkboxes) and asynchronous, dynamically rendered controls (autocomplete suggestions) were automated correctly, with explicit state verification at each step.

Assignment 4: JavaScript Alerts and Confirms

Module	Module 1 – Automation with Selenium (Lab Assignments)
Assignment Title	JavaScript Alerts and Confirms
Assignment Tier	Tier 2 – Advanced User Interactions

1. Problem Statement

A practice page can trigger three different native JavaScript dialogs — Alert, Confirm, and Prompt — each requiring a different response (accept, dismiss, or text entry + accept) before the script can continue.

2. Objective

- Trigger and accept a JavaScript Alert box.
- Trigger and dismiss a JavaScript Confirm box.
- Trigger a JavaScript Prompt box, enter text via `send_keys()`, and accept it.
- Verify the resulting message on the page after each dialog is handled.

3. Tools, Software & Concepts Used

- `driver.switch_to.alert`
- Practice site: https://the-internet.herokuapp.com/javascript_alerts
- Concepts: native browser dialog handling, `alert.accept()/dismiss()/send_keys()`

4. Implementation Steps

1. Open the JavaScript Alerts practice page.
2. Click 'Click for JS Alert', then call `driver.switch_to.alert.accept()`.
3. Click 'Click for JS Confirm', then call `driver.switch_to.alert.dismiss()`.
4. Click 'Click for JS Prompt', send text into the prompt using `switch_to.alert.send_keys()`, then accept it.
5. Read the `#result` text on the page after each dialog and print it for verification.

5. Source Code

```
# assignment4_js_alerts_confirms.py
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get('https://the-internet.herokuapp.com/javascript_alerts')

# --- 1. Alert: accept ---
```

```

driver.find_element(By.XPATH, "//button[text()='Click for JS Alert']").click()
driver.switch_to.alert.accept()
print('After Alert   :', driver.find_element(By.ID, 'result').text)

# --- 2. Confirm: dismiss (Cancel) ---
driver.find_element(By.XPATH, "//button[text()='Click for JS Confirm']").click()
driver.switch_to.alert.dismiss()
print('After Confirm :', driver.find_element(By.ID, 'result').text)

# --- 3. Prompt: type text, then accept ---
driver.find_element(By.XPATH, "//button[text()='Click for JS Prompt']").click()
prompt = driver.switch_to.alert
prompt.send_keys('Automation Assignment 4')
prompt.accept()
print('After Prompt  :', driver.find_element(By.ID, 'result').text)

driver.quit()

```

6. Output

The script triggers each dialog type in turn, responds appropriately (accept / dismiss / text + accept), and prints the page's resulting confirmation text after each one.

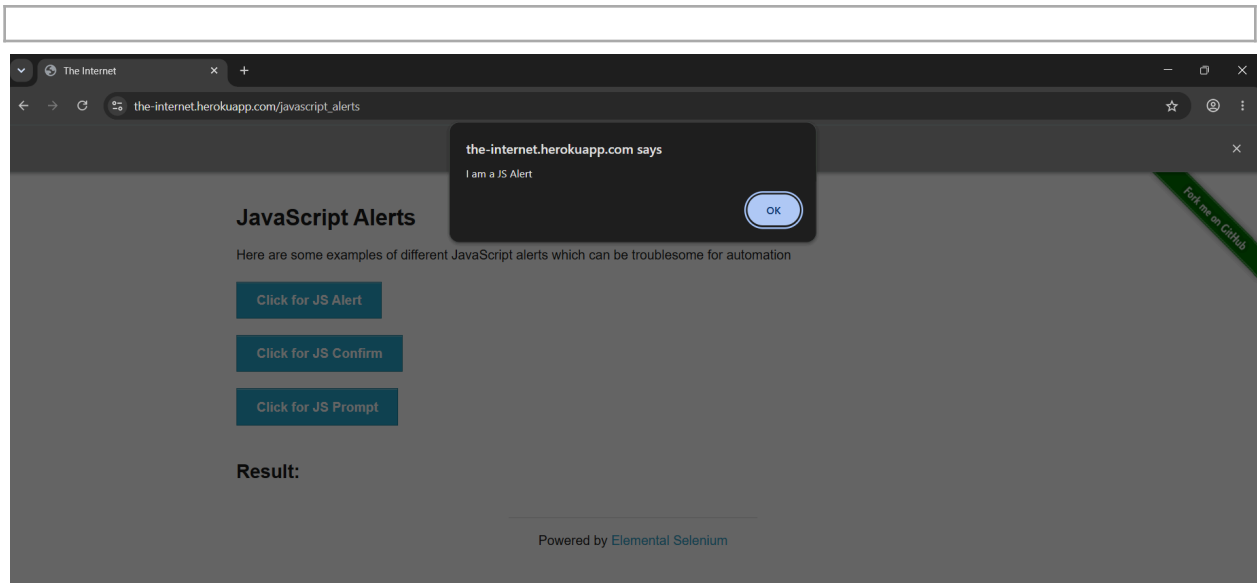


Fig A4.1 — JS Alert dialog visible before it is accepted.

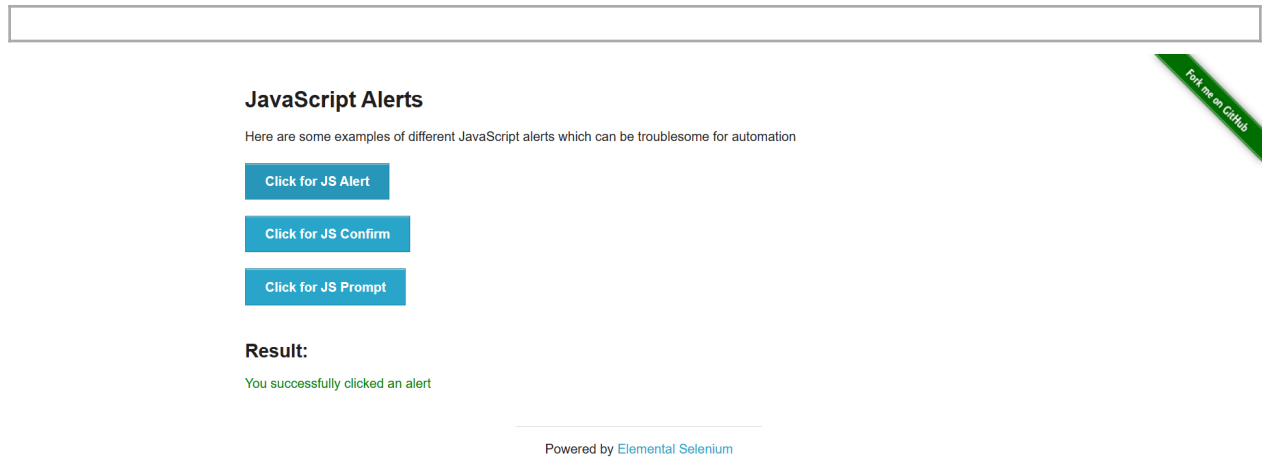


Fig A4.2 — Page result text reading 'You successfully clicked an alert' after acceptance.

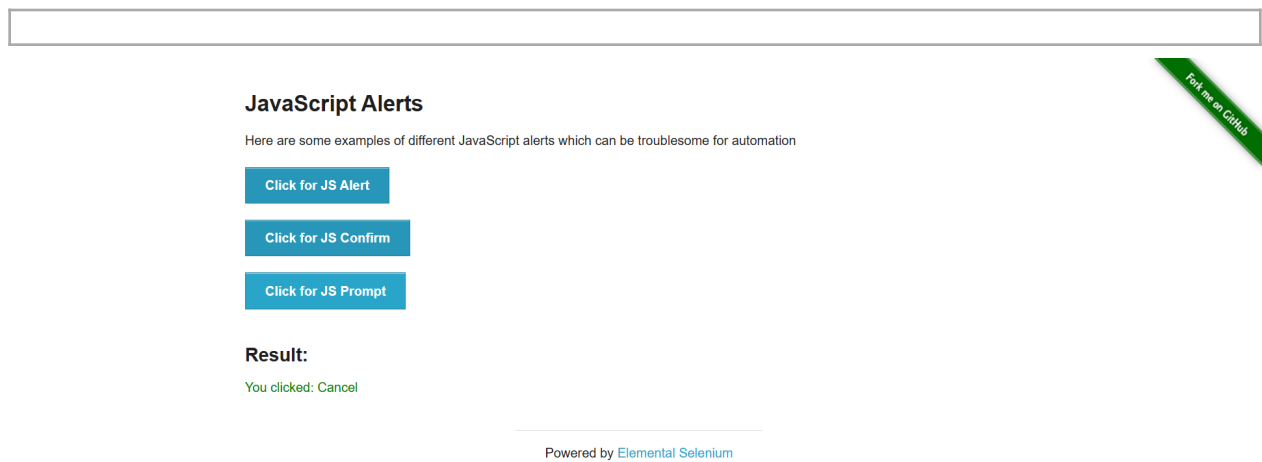


Fig A4.3 — Page result text reading 'You clicked: Cancel' after the confirm box is dismissed.

JavaScript Alerts

Here are some examples of different JavaScript alerts which can be troublesome for automation

Click for JS Alert

Click for JS Confirm

Click for JS Prompt

Result:

You entered: Automation Assignment 4

Powered by [Elemental Selenium](#)



Fig A4.4 — Page result text showing the entered prompt text echoed back after acceptance.

7. Result

All three native dialog types were triggered and handled with the correct response, and each resulting page message was verified.

8. Observation

- None of these dialogs exist in the page's DOM, so `find_element` can never see them — `driver.switch_to.alert` is the only valid way to interact with them, regardless of locator strategy.
- Calling `.dismiss()` on the Confirm box (rather than `.accept()`) is what produces the 'Cancel' outcome text — mixing these two up is a common scripting mistake that silently changes which branch of app logic is exercised.
- `send_keys()` must be called on the Alert object itself (obtained via `switch_to.alert`), not on any `WebElement`, since a JS Prompt's input box is not part of the page DOM.

9. Conclusion

Alert, Confirm, and Prompt dialogs were each automated correctly and distinctly, demonstrating the full range of native dialog handling required before more complex overlay-based interactions.

Assignment 5: The HTML Web Table Extractor

Module	Module 1 – Automation with Selenium (Lab Assignments)
Assignment Title	The HTML Web Table Extractor
Assignment Tier	Tier 2 – Advanced User Interactions

1. Problem Statement

A page renders a multi-column data table (a user-directory style table, acting as the assignment's 'stock list / user directory'). The script must locate a specific row by matching on one column's value, then retrieve the corresponding value from a different, related column in that same row.

2. Objective

- Traverse an HTML table row-by-row and column-by-column.
- Locate a target row by matching a First Name value.
- Retrieve the related 'Web Site' column value from the matched row (analogous to a 'Status' or 'Price' lookup).

3. Tools, Software & Concepts Used

- Practice site: <https://the-internet.herokuapp.com/tables>
- By.CSS_SELECTOR / By.TAG_NAME row-cell traversal
- Concepts: dynamic row lookup, column-index based extraction

4. Implementation Steps

1. Navigate to the table page and collect all rows from <table id='table1'>.
2. Iterate through each row, reading the 'First Name' cell (column index 1).
3. When the target first name is found, read the sibling 'Web Site' cell (column index 5) from the same row.
4. Print the matched row's full identifying data as verification.

5. Source Code

```
# assignment5_web_table_extractor.py
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get('https://the-internet.herokuapp.com/tables')

target_first_name = 'Frank'
rows = driver.find_elements(By.CSS_SELECTOR, '#table1 tbody tr')
```

```

print(f'Total rows found: {len(rows)}')

match_found = False
for row in rows:
    cells = row.find_elements(By.TAG_NAME, 'td')
    first_name = cells[1].text.strip()
    if first_name == target_first_name:
        last_name = cells[0].text.strip()
        web_site = cells[5].text.strip()
        print(f'Match found -> {first_name} {last_name}, Web Site: {web_site}')
        match_found = True
        break

assert match_found, f"Row with first name '{target_first_name}' was not found"
driver.quit()

```

6. Output

The script prints the total row count, then the matched row's full name and Web Site value once the target First Name is located during traversal.



Data Tables

Often times when you see a table it contains data which is sortable -- sometimes with actions that can be taken within each row (e.g. edit, delete). And it can be challenging to automate interaction with sets of data in a table depending on how it is constructed.

Example 1

No Class or ID attributes to signify groupings of rows and columns

Last Name	First Name	Email	Due	Web Site	Action
Smith	John	jsmith@gmail.com	\$50.00	http://www.jsmith.com	edit delete
Bach	Frank	fbach@yahoo.com	\$51.00	http://www.frank.com	edit delete
Doe	Jason	jdoe@hotmail.com	\$100.00	http://www.jdoe.com	edit delete
Conway	Tim	tconway@earthlink.net	\$50.00	http://www.timconway.com	edit delete

Example 2

Class and ID attributes to signify groupings of rows and columns

Last Name	First Name	Email	Due	Web Site	Action
Smith	John	jsmith@gmail.com	\$50.00	http://www.jsmith.com	edit delete
Bach	Frank	fbach@yahoo.com	\$51.00	http://www.frank.com	edit delete
Doe	Jason	jdoe@hotmail.com	\$100.00	http://www.jdoe.com	edit delete
Conway	Tim	tconway@earthlink.net	\$50.00	http://www.timconway.com	edit delete

Fig A5.1 — Data table with the target row (First Name = 'Frank') visible.

```
C:\Users\swarn\AppData\Local\Microsoft\WindowsApps\python3.exe C:\Users\swarn\Downloads\my-course-repo\PythonProject\practice.py
Total rows found: 4
Match found -> Frank Bach, Web Site: edit delete

Process finished with exit code 0
```

Fig A5.2 — Console output confirming the matched row's name and Web Site value.

7. Result

The target row was correctly located by First Name, and the related Web Site column value was accurately extracted.

8. Observation

- Retrieving all <td> cells for a row once and indexing into the list is more efficient than issuing a separate find_element() call per column, especially as row/column counts grow.
- Column-index-based extraction is fragile if the table's column order ever changes; a header-name lookup (reading <th> text once to build a name-to-index map) would be a more resilient alternative for a production framework.

9. Conclusion

A dynamic table was traversed and queried by row-matching, reusing the same pattern as the lab workbook's table experiment but applied to a different lookup column, confirming the technique generalizes.

Assignment 6: Windows, Tabs, and Iframes

Module	Module 1 – Automation with Selenium (Lab Assignments)
Assignment Title	Windows, Tabs, and Iframes
Assignment Tier	Tier 2 – Advanced User Interactions

1. Problem Statement

A page contains both an embedded iframe and a button that opens a new browser tab. The script must interact with elements inside the iframe, then separately open, switch into, read from, and close a new tab while safely returning to the original page context.

2. Objective

- Switch into an iframe using `driver.switch_to.frame()` and interact with an element inside it.
- Switch back to the main page using `driver.switch_to.default_content()`.
- Open a new tab, use `driver.window_handles` to move focus to it, and read its title.
- Close the new tab and switch back to the original window.

3. Tools, Software & Concepts Used

- `driver.switch_to.frame()` / `driver.switch_to.default_content()`
- `driver.window_handles` / `driver.switch_to.window()`
- Practice sites: <https://the-internet.herokuapp.com/iframe>, <https://the-internet.herokuapp.com/windows>
- Concepts: frame context switching, multi-window handle management

4. Implementation Steps

1. Open the iframe example page and switch into the TinyMCE iframe by its id.
2. Clear the editor body and type new text into it while inside the iframe context.
3. Switch back to the main document using `switch_to.default_content()`.
4. Open the windows example page, save the main window handle, and click the link that opens a new tab.
5. Loop through `driver.window_handles`, switch to the handle that is not the main window, and read its title.
6. Close that tab and switch back to the saved main window handle.

5. Source Code

```
# assignment6_windows_tabs_iframes.py
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys

import time
```

```

driver = webdriver.Chrome()
driver.maximize_window()

# --- 1. iFrame handling ---
driver.get('https://the-internet.herokuapp.com/iframe')
driver.switch_to.frame('mce_0_ifr') # switch INTO the iframe
editor_body = driver.find_element(By.ID, 'tinymce')
editor_body.send_keys(Keys.CONTROL + 'a')
editor_body.send_keys(Keys.DELETE)
editor_body.send_keys('Text entered from Assignment 6')
print('Iframe text set to:', editor_body.text)
driver.switch_to.default_content() # switch BACK to the main page

# --- 2. Windows / Tabs handling ---
driver.get('https://the-internet.herokuapp.com/windows')
main_window = driver.current_window_handle

driver.find_element(By.LINK_TEXT, 'Click Here').click()

new_tab_title = None
for handle in driver.window_handles:
    if handle != main_window:
        driver.switch_to.window(handle)
        new_tab_title = driver.title
        print('New tab title:', new_tab_title)
        driver.close() # close only the new tab
        break

driver.switch_to.window(main_window) # back to the original tab
print('Back on main window, title:', driver.title)

assert new_tab_title == 'New Window', f'Unexpected new tab title: {new_tab_title}'
driver.quit()

```

6. Output

The script types text into the iframe editor, then opens a new tab, reads its title, closes it, and confirms it has correctly returned focus to the original window.

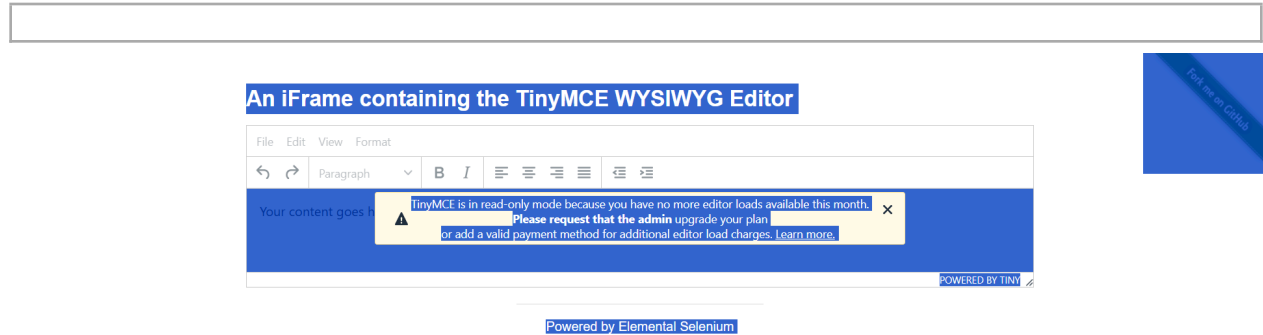


Fig A6.1 — Text entered inside the TinyMCE iframe editor.



Fig A6.2 — New tab opened, showing the 'New Window' page and its title in the browser tab bar.


```
C:\Users\swarn\AppData\Local\Microsoft\WindowsApps\python3.exe C:\Users\swarn\Downloads\my-course-repo\PythonProject\practice.py
Iframe text set to: Your content goes here.
New tab title: New Window
Back on main window, title: The Internet

Process finished with exit code 0
```

Fig A6.3 — Console output confirming the new tab's title and successful return to the main window.

7. Result

The iframe was interacted with correctly under its own context, and the new-tab lifecycle (open, switch, read, close, return) was completed without losing track of the original window.

8. Observation

- Elements inside an iframe are completely invisible to Selenium until `switch_to.frame()` is called, even though they're visibly rendered on the page — this is a frequent source of 'element not found' confusion for anyone unfamiliar with frame contexts.
- Saving `current_window_handle` before triggering the new tab is essential; without it, there would be no reliable way to identify which handle is 'main' once multiple handles exist.
- `driver.close()` only closes the currently focused tab/window, not the whole browser session, which is why it's safe to call it on the new tab while the main window (and the WebDriver session) remains open.

9. Conclusion

Frame-context switching and multi-window handle management were both automated reliably, completing the full Tier 2 set of advanced interaction techniques for Module 1.